03

Domínio de Eloquent Avançado

Domínio de Eloquent Avançado

Evolução do projeto dia1 com relacionamentos, consultas eficientes e filtros reutilizáveis.

Roteiro da aula

- Problema real no projeto dia1
- Revisão rápida de relacionamentos
- Relacionamentos complexos
- MorphToMany
- HasManyThrough
- Pivot customizado
- O problema N+1
- Eager loading dinâmico
- Lazy eager loading

Problema real no projeto dia1

Cenário
A listagem de produtos precisa mostrar categoria, fornecedores, tags e dados calculados sem multiplicar consultas.

Problemas comuns
- Muitas queries para montar uma única tela.
- Relacionamentos mal definidos.
- Filtros repetidos em vários controllers.
- O model Product acumula métodos de consulta.

Revisão rápida de relacionamentos

Relacionamentos básicos

- belongsTo: um produto pertence a uma categoria.
- hasMany: uma categoria possui muitos produtos.
- belongsToMany: um produto pode ter vários fornecedores.

Exemplo

Exemplo de código

```
class Product extends Model
{
    public function category(): BelongsTo
    {
        return $this->belongsTo(Category::class);
    }
}
```

Relacionamentos complexos

Tópicos obrigatórios

- MorphToMany
- HasManyThrough
- modelos de Pivot customizados

Ideia

Relacionamentos complexos ajudam o código a expressar o domínio quando a estrutura do banco deixa de ser simples.

MorphToMany

Cenário
Produtos e categorias podem receber tags.

Exemplo

Uso

- tags;
- anexos;
- comentários;
- atributos reutilizados por vários tipos de entidade.

Exemplo de código

```
class Product extends Model
{
    public function tags(): MorphToMany
    {
        return $this->morphToMany(Tag::class, 'taggable');
    }
}
```

HasManyThrough

Cenário
Uma categoria quer acessar avaliações de todos os seus produtos.

Exemplo conceitual

Cuidado
Use quando a relação indireta realmente existir no modelo de dados.

Exemplo de código

```
class Category extends Model
{
    public function reviews(): HasManyThrough
    {
        return $this->hasManyThrough(
            ProductReview::class,
            Product::class,
        );
    }
}
```

Pivot customizado

Cenário
Um produto possui fornecedores, mas cada vínculo guarda preço negociado e status.

Exemplo

Uso no relacionamento

Exemplo de código

```
class ProductSupplier extends Pivot
{
    protected $casts = [
        'price' => 'decimal:2',
        'active' => 'boolean',
    ];
}
```

O problema N+1

Exemplo ruim

Explicação

Uma query busca produtos. Depois, cada linha pode gerar outra query para buscar a categoria.

Solução

Exemplo de código

```
$products = Product::all();

foreach ($products as $product) {
    echo $product->category->name;
}
```

Eager loading dinâmico

Cenário
A API pode receber ?include=category,tags,suppliers.

Exemplo

Cuidado
Nunca aceite qualquer relacionamento enviado pelo usuário sem uma lista permitida.

Exemplo de código

```
$includes = collect(explode(',', request('include', '')))
    ->intersect(['category', 'tags', 'suppliers'])
    ->all();

$products = Product::query()
    ->with($includes)
    ->paginate();
```

Lazy eager loading

Uso
Quando os produtos já foram carregados e a aplicação decide depois quais relações carregar.

Exemplo

Ponto didático
Não é desculpa para consultar sem pensar. É uma ferramenta para casos em que a decisão acontece depois.

Exemplo de código

```
$products = Product::latest()->take(20)->get();

$products->loadMissing(['category', 'tags']);
```

Subqueries avançadas

Cenário
Listar produtos com a data da avaliação mais recente.

Exemplo

Uso

- rankings;
- última avaliação;
- menor preço de fornecedor;
- contadores calculados.

Exemplo de código

```
Product::query()
    ->addSelect([
        'last_review_at' => ProductReview::query()
            ->select('created_at')
            ->whereColumn('product_id', 'products.id')
            ->latest()

            ->limit(1),
    ])
    ->get();
```

Custom Builders

Problema
O model Product acumula muitos métodos de filtro.

Solução
Criar Builder dedicado.

Exemplo de código

```
final class ProductBuilder extends Builder
{
    public function priceBetween(float $min, float $max):
self
    {
        return $this->whereBetween('price', [$min, $max]);
    }

    public function inCategory(int $categoryId): self
    {
        return $this->where('category_id', $categoryId);
    }
}
```

Projeto do dia

Missão
Evoluir a listagem de produtos do projeto dia1.

Entregas
- Modelar categorias, fornecedores e tags polimórficas.
- Criar pivot customizado entre produtos e fornecedores.
- Usar eager loading para evitar N+1.
- Criar subquery para a última avaliação.
- Criar ProductBuilder para filtros comuns.

Checklist final

O aluno deve conseguir

- Escolher relacionamento Eloquent adequado.
- Explicar MorphToMany, HasManyThrough e Pivot customizado.
- Identificar N+1.
- Aplicar eager loading dinâmico com lista permitida.
- Usar loadMissing.
- Criar subquery com Eloquent.
- Mover filtros para ProductBuilder.